Python Engineering - Class 10

Funciones avanzadas | Guía de ejercicios

Objetivo de la clase: Comprender distintas formas de pasar argumentos, desempaquetar estructuras y tratar funciones como objetos.

Este documento es la guía base del ejercicio. No contiene una solución completa para copiar: organiza el problema, los pasos de implementación, los valores de referencia y la validación.

Ruta

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/class_10_advanced functions/
    |-- notes/
    `-- exercices/
```

Flujo de trabajo obligatorio

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Antes de programar, redacta estas secciones. El objetivo es separar el razonamiento de la sintaxis.

EX18 - Argumentos avanzados y funciones como objetos

Problema: Practicar argumentos posicionales, keyword, defaults, *args, **kwargs, unpacking y funciones pasadas como argumento.

Entradas de referencia: diccionarios config, secuencias de medidas y funciones add/multiply

Ecuaciones / relación: energía específica, medias y operaciones simples

Comprobación esperada: *args es tuple; **kwargs es dict; las keys de **config deben coincidir con parámetros; distinguir add de add().

Guía de implementación

- 1. Llamar con argumentos posicionales/keyword.
- 2. Usar default g.
- 3. Crear *args.
- 4. Crear **kwargs.
- 5. Desempaquetar list/dict.
- 6. Pasar add o multiply a apply_operation.

Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

Validación final

Responde estas preguntas como comentarios al final del ejercicio. El objetivo es demostrar que puedes explicar el flujo del código.

- Diferencia entre argumento posicional y keyword.
- Que ocurre si no pasamos un argumento con default?
- Que tipo es *args dentro de la funcion?
- Que tipo es **kwargs?
- Que significa f(*values)?
- Por que las keys de config deben coincidir con parametros en f(**config)?
- Diferencia entre add y add().

Criterio de cierre

La clase se considera completada cuando el código se ejecuta, las comprobaciones de referencia tienen sentido, puedes explicar cada bloque sin leer una solución y puedes modificar al menos un input anticipando su efecto.